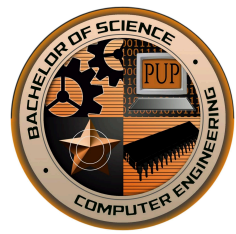




Republic of the Philippines
Polytechnic University of the Philippines - Sta. Mesa
Department of Computer Engineering
College of Engineering and Architecture



INTERide: An Object-Oriented Ride Booking System using Python and Tkinter

Submitted by:

Aragoncillo, Mandy Nicole B.

Calasang, Gabriel Carl S.

Caspillan, Rudylyn S.

Laguine, Samantha Shan A.

Lemindog, Manchor Jr. D.

Maico, Princess Venice N.

Murillo, Marian Andrea L.

Racanday, Gian Gabriel A.

Tapado, Dorothy Ane C.

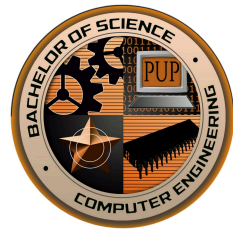
Submitted to:

Engr. Avena, Godofredo T.
Object-Oriented Programming Instructor

June 15, 2026



Republic of the Philippines
Polytechnic University of the Philippines - Sta. Mesa
Department of Computer Engineering
College of Engineering and Architecture



ABSTRACT

INTERide is a desktop-based ride booking system developed as an academic project, under the course of Object-Oriented Programming, by the students from the Department of Computer Engineering at Polytechnic University of the Philippines. Built with the use of Python and Tkinter, designed to demonstrate the practical application of the course principles in a real-world context.

The system incorporates the four core pillars of OOP — inheritance, encapsulation, polymorphism, and abstraction — through a structured class hierarchy consisting of a base Vehicle class and its subclasses (Car, Van, and Bike), each with distinct fare computation behavior. Users can register an account, book a ride by entering their pickup and drop-off locations, select a vehicle type, and receive a computed fare estimate based on actual road distances retrieved via the OSRM (Open Source Routing Machine) API. Address inputs are geocoded using “geopy”, and route paths are visualized through an interactive map widget powered by “tkintermapview” and “OpenStreetMap”.

Booking records are stored persistently in a local JSON file and can be exported as formatted Word document receipts using python-docx. The system also supports multiple payment options including cash and GCash. Testing confirmed that all core functionalities — booking creation, fare calculation, data storage, and receipt generation — performed as intended.

INTERide demonstrates how academic OOP concepts can be applied to simulate a functional ride-hailing platform. The project reflects both the technical relevance of software design principles and the social importance of accessible transportation solutions in the Philippine context.

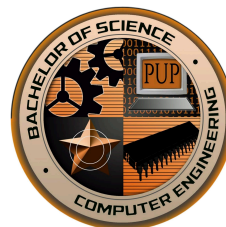
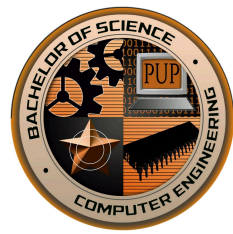


TABLE OF CONTENTS

	Page
Title Page	
Abstract.....	1
Table of Contents.....	2
I. INTRODUCTION	
1.1 Background.....	3-4
1.2 Problem Statement.....	4
1.3 Project Objectives.....	5
1.4 Scope.....	5
II. METHODOLOGY	
2.1 Research Design.....	6-8
2.2 Data Collection Methods.....	8
2.3 Tools Used.....	8
III. SYSTEM DESIGN AND IMPLEMENTATION	
3.1 System Overview.....	9-10
3.2 Key Components.....	10-11
3.3 Implementation Process.....	11
IV. TESTING AND RESULTS	
4.1 Testing Methods.....	12
4.2 Test Results.....	13-16
4.3 Analysis of Results.....	17
V. CONCLUSION	
5.1 Summary of Findings.....	18-19
5.2 Recommendations.....	19-20



I. INTRODUCTION

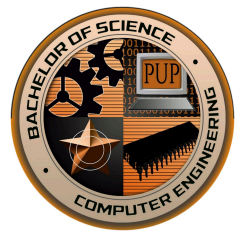
1.1 Background

In recent years, ride-booking services such as MoveIt, Angkas, and Grab have become important components of modern urban transportation systems. These platforms function by connecting commuters with drivers through mobile applications, allowing users to input their pick-up and drop-off locations, view fare estimates, and track their rides in real time. This system streamlines the commuting process by reducing uncertainty and providing transparency in both pricing and travel routes, making it a practical alternative to traditional public transport.

The relevance of ride-booking platforms in contemporary society stems from their capacity to mitigate critical urban issues such as traffic congestion and insufficient public transport systems. As the demand for faster and more efficient mobility solutions continues to rise, these applications provide commuters with dependable access to timely transportation. Furthermore, they play a crucial role in advancing the modernization of transit infrastructure by embedding digital technologies into everyday travel practices, thereby improving both accessibility and overall convenience.

Moreover, Ride-booking apps also add extra safety and reliability for commuters. With features like driver ratings, GPS tracking, and cashless payments, passengers can feel more secure during their trips. Sharing ride details with family or friends further increases protection, while clear fare information helps avoid unfair charges. Altogether, these tools make commuting smoother and safer, especially in busy cities where both security and efficiency are very important.

Beyond commuter benefits, these platforms also generate economic opportunities by offering flexible employment for drivers and service providers. This dual impact—supporting livelihoods while improving mobility—shows the broader value of ride-booking systems. They are not just convenience tools but essential elements of modern urban life, reshaping



how people move through cities and contributing to the development of sustainable transportation systems.

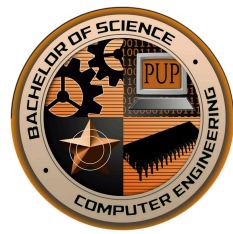
However, most existing ride-booking applications remain limited to specific vehicle types, such as motorcycles or cars. This restricts commuters who may prefer other modes of transport depending on their destination, budget, or comfort level. The lack of variety often forces users to switch between different apps, which can be inconvenient and time-consuming. To address this gap, our proposed ride-booking system introduces a more versatile platform that allows commuters to book multiple types of vehicles within a single application. Fares will be calculated based on the destination and chosen vehicle type, ensuring flexibility and convenience. This system aims to deliver a comprehensive, all-in-one ride-booking experience tailored to the diverse needs of commuters.

1.2 Problem Statement

While ride-booking applications have improved commuting, several challenges remain in existing systems:

- Limited vehicle choices reduce commuter flexibility.
- Fare computations are often uniform and fail to reflect differences in vehicle type or efficiency.
- Insufficient clarity in fare breakdowns, including tolls, taxes, and fuel costs.
- The need to switch between multiple apps for different transport modes causes inconvenience.
- Weak data management can result in booking errors or delays.
- Limited features for saving and monitoring bookings.

These challenges emphasize the necessity of a more integrated and systematic ride-booking platform—one that can offer diverse vehicle options, accurate and transparent fare calculations, efficient data handling, and improved tools for trip tracking. Addressing these



gaps will enhance commuter satisfaction and support the development of a more reliable urban mobility system.

1.3 Project Objective

1. To establish a flexible system architecture that applies Object-Oriented Programming (OOP) concepts such as inheritance, encapsulation, and polymorphism.
2. To design and implement a booking interface that enables commuters to reserve, monitor, and complete rides with ease.
3. To incorporate multi-vehicle management, allowing different transport types to be represented with unique attributes such as fare rate, seating capacity, and efficiency.
4. To develop booking functionalities that include ride selection, accurate fare computation, trip history viewing, and cancellation options.
5. To integrate persistent data handling through file operations, ensuring that booking records can be saved, retrieved, and maintained across sessions.
6. To build an intuitive graphical user interface (GUI) using Tkinter that promotes smooth and user-friendly interaction.

1.4 Scope

The scope of this study focuses on the design and development of a ride-booking system that integrates multiple vehicle types and modern features to enhance the commuter experience. It includes the creation of a system architecture that ensures flexibility and scalability. The study also covers the development of a booking interface that allows users to reserve, track, and complete rides, along with vehicle management that supports diverse transport modes such as motorcycles, cars, and vans—each defined by unique attributes like fare rate and capacity. Core functionalities such as ride selection, fare computation, trip history viewing, and cancellation are incorporated to provide a complete booking process. In addition, file operations are integrated to enable saving and loading of booking records, ensuring data persistence across sessions. A user-friendly graphical interface using Tkinter is developed to promote smooth and intuitive interaction.

II. METHODOLOGY

2.1 Research Design

The INTERide booking system is designed with concepts of Object Oriented Programming as the primary methodological framework for the structure and organization of the logic in the system. The graphical user interface employed is the Tkinter, developed and designed for user interaction such as entering information, and clicking buttons for submitting or cancelling bookings, and a display section for viewing all the bookings made.

Inheritance concept is demonstrated by implementing the vehicle base class that contains shared attributes and behavior then the subclasses car, van, and bike.

```
14 class Vehicle:
15     def __init__(
16         self,
17         vtype: str,
18         cap: int,
19         rate: float,
20         emoji: str,
21         tagline: str,
22     ) -> None:
23         self._vtype: str = vtype
24         self._cap: int = cap
25         self._rate: float = rate
26         self._emoji: str = emoji
27         self._tagline: str = tagline
```

Figure 1.1. Vehicle Base Class

```
48 class Car(Vehicle):
49     def __init__(self) -> None:
50         super().__init__("Car", 4, 15.0, "🚗", "Comfortable · 4 seats")
```

Figure 1.2. Car Subclass

```
56 class Van(Vehicle):
57     def __init__(self) -> None:
58         super().__init__("Van", 10, 25.0, "🚐", "Spacious · 10 seats")
59
```

Figure 1.3. Van Subclass

```
64 class Bike(Vehicle):
65     def __init__(self) -> None:
66         super().__init__("Bike", 1, 8.0, "🚲", "Fast · 1 rider")
67
```

Figure 1.4. Bike Subclass

Encapsulation is applied by making the attributes private to protect the internal properties and only show them through read only attributes.

```
93     self._bid: str = bid or str(uuid.uuid4())[8].upper()
94     self._user: str = user
95     self._v: Vehicle = V_MAP[vtype]
96     self._pick: str = pickup
97     self._drop: str = dropoff
98     self._pc: Tuple[float, float] = pc
99     self._dc: Tuple[float, float] = dc
100     self._km: float = round(km, 2)
101     self._cost: float = self._v.calculate_cost(km)
102     self._ts: str = ts or datetime.now().strftime("%Y-%m-%d %H:%M")
```

Figure 1.5. Example Application of Encapsulation

To override the cost calculation method for each vehicle type, Polymorphism is integrated.

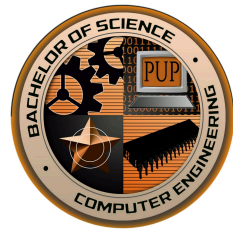
```
48 class Car(Vehicle):
49     def __init__(self) -> None:
50         super().__init__("Car", 4, 15.0, "🚗", "Comfortable · 4 seats")
51
52     def calculate_cost(self, km: float) -> float:
53         return round(self._rate * km * (0.90 if km > 10 else 1.0), 2)
54
```

Figure 1.6. Example Cost Calculation Override in the Car Subclass

With the object oriented structure, the system is ensured with modularity and reusability with each class using real objects and concluding its own data and behavior which is aligned with the approach of Object-Oriented Programming

2.2 Data Collection

Direct user interaction with the graphical user interface is the data collection method in this system. When a user books a ride, they are required to enter their name, select their vehicle type, and input their pickup and drop-off location of their ride.



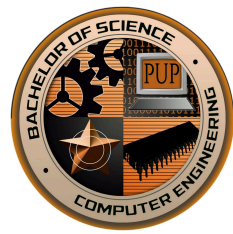
Geocoding (geo.py) is integrated as the typed address string by the user is converted into geographic coordinates to improve search accuracy. The user's general location is detected through IP-based geolocation as a default.

The OSRM (Open Source Routing Machine) API is integrated, which provides actual road-network data based on the pickup and drop-off coordinates, and is used to gather route distance and estimated journey time.

After a ride booking is confirmed, all the collected details such as the booking ID, passenger name, vehicle of choice, locations, distance, estimated fare, it is compiled into a booking object and converted into a local JSON file (interide_bookings.json) for storage. This file accumulates one record for each transaction that is successfully completed, providing it as the program's dataset.

2.3 Tools Used

The primary programming language used in the system is *Python* and *Tkinter* as the Graphical User Interface framework for designing interfaces. The system incorporates *geopy* with the geocoding tool to convert addresses into coordinates, *tkintermapview* to display OpenStreetMap tiles, and the *OSRM public API* to determine accurate driving routes, distances, and durations for mapping and geolocation. The bookings file is read and written using *Python's built-in json library* for data persistence, and prepared Word document receipts with tables, styled text, and color formatting are produced using python-docx. After that, the *subprocess and importlib* were used to develop a self-installing launcher (run.py) that ensures the program operates with minimal manual setup by automatically checking for and installing any missing dependencies.



III. SYSTEM DESIGN AND IMPLEMENTATION

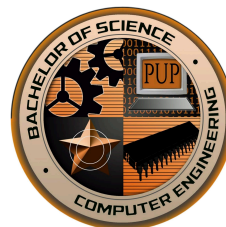
3.1 System Overview

Ride-booking systems are digital platforms that connect passengers with drivers through a centralized application. In the Philippines, services such as Grab, Angkas, and JoyRide have become popular because they simplify commuting by allowing users to book rides, estimate fares, and track trips without the need for manual arrangements. These systems help address urban mobility challenges, particularly in congested areas like Metro Manila, by offering commuters safer and more predictable travel options.

The INTERide Ride Booking System was developed as a desktop application using Python and Tkinter. It integrates multiple functions such as user registration, booking creation, fare estimation, and record management into one cohesive platform. Unlike traditional manual transport arrangements, INTERide reduces errors and delays by storing booking data in a structured MySQL database. This ensures that ride transactions are properly documented and retrievable, reflecting the importance of information systems in transportation management.

A key advantage of INTERide is its flexibility in vehicle selection. While many academic prototypes limit options to a single transport type, INTERide supports cars, vans, and bikes, allowing users to choose based on passenger capacity and travel distance. This mirrors real-world practices where commuters select motorcycles for short trips or vans for group travel. The system also supports multiple payment methods, including cash and GCash, aligning with the growing adoption of cashless transactions in the Philippines.

Beyond booking, INTERide emphasizes transaction transparency. Each ride generates a record that can be stored, retrieved, and even exported as a receipt. This feature not only improves accountability but also provides commuters with proof of payment, similar to digital receipts used in Grab and JoyRide. By combining transportation planning, fare computation, and record management, INTERide demonstrates how academic projects can simulate real-world transport solutions.



Overall, INTERide showcases the integration of programming concepts with practical transport needs. It highlights how Object-Oriented Programming principles such as modular models for users, bookings, and ratings can be applied to build a functional ride-booking system. The project reflects both the technical and social relevance of ride-booking applications in modern commuting.

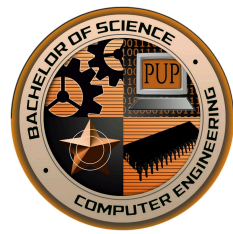
3.2 Key Components

COMPONENTS	FUNCTION/PURPOSE
Database Layer	Establishes and manages the connection to the MySQL database. It is responsible for creating tables, executing queries, and ensuring data persistence. This layer guarantees that user accounts, bookings, and ratings are securely stored and retrievable for future transactions.
User Model	Handles all user-related operations, including account creation, login validation, and profile updates. It incorporates password hashing to strengthen security and prevent unauthorized access. By centralizing user management, the system ensures that personal information is consistently maintained.
Booking Model	Manages the booking process by recording trip details such as pickup and drop-off locations, vehicle type, fare, and payment method. It also updates booking statuses and retrieves booking histories, allowing users to track past and ongoing rides. This model reflects the transactional core of the system.
Rating Model	Provides a mechanism for passengers to evaluate their ride experience. Ratings and feedback are stored in the database and linked to specific bookings, supporting accountability and service improvement.

Map Widget	A Tkinter-based interactive canvas that visually represents routes, pickup points, and drop-off locations. It enhances user experience by offering a graphical view of the journey, making the booking process more intuitive.
Graphical User Interface Screens	Includes the splash, login, registration, and dashboard screens. These interfaces provide structured navigation and ensure that users can interact with the system in a clear and organized manner. Each screen is designed to guide the user through specific tasks such as account creation, ride booking, and transaction review.

3.3 Implementation Process

- 01 ● **Plan the system design**
The team prepared GUI mockups, defined color palettes, and outlined functional requirements such as user registration, booking creation, and fare calculation.
- 02 ● **Set up the database**
A MySQL database named 'interide_db' was created, with tables for users, bookings, and ratings auto-generated on first launch.
- 03 ● **Develop core modules**
Object-Oriented Programming principles were applied to build modular classes for users, bookings, and ratings, ensuring maintainability and scalability.
- 04 ● **Integrate the GUI**
Tkinter was used to implement login, registration, and dashboard screens, connecting user inputs with backend logic.
- 05 ● **Add interactive map feature**
A custom Tkinter canvas widget was developed to visualize pickup and drop-off points, routes, and driver pins.
- 06 ● **Test and debug functions**
Unit and integration testing were performed to validate login, fare computation, booking storage, and GUI responsiveness.
- 07 ● **Deploy the application**
Dependencies such as mysql-connector-python and Pillow were installed, and the system was packaged for operational use.



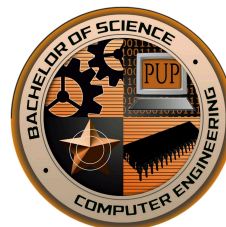
IV. TESTING AND RESULTS

4.1 Testing Methods

The Testing and Results phase was conducted to validate the functionality, usability, and reliability of the INTERide Ride Booking System. This stage involved systematic verification of the system's core modules, ranging from database operations and user authentication to fare calculation and graphical interface responsiveness. Each test was designed to confirm that the application performs as intended and aligns with the objectives of a ride-booking platform.

The system was tested using several techniques:

- **Unit Testing** – Verified individual functions such as user login, fare calculation, and booking creation. For example, the UserModel was tested to ensure proper credential validation and password hashing, while the BookingModel was tested for accurate fare computation based on distance and vehicle type
- **Integration Testing** – Checked interactions between modules, particularly the connection between Tkinter GUI inputs and MySQL database queries. This confirmed that user actions in the interface were correctly reflected in the database tables (users, bookings, ratings).
- **User Acceptance Testing** – Simulated real booking scenarios to confirm usability and accuracy. Test users created accounts, booked rides with different vehicles, and generated receipts to validate the end-to-end process.
- **Validation Testing** – Ensured proper handling of invalid inputs, such as empty fields, duplicate usernames, or incorrect email formats. This was critical for maintaining data integrity and preventing transaction errors.
- **Performance Testing** – Measured response times for database queries and GUI rendering. The system was tested under multiple booking requests to ensure responsiveness and stability.



4.2 Test Results

The outcomes of the testing phase confirmed that INTERide successfully met its functional requirements:

A screenshot of the INTERide web application's login page. The page has a blue header with the "INTERide" logo and a "Welcome back! Please log in." message. Below the header is a white box with the title "Login to your account" and a subtitle "Enter your username and password to continue." Inside this box are two input fields: "Username" with the text "micho" and "Password" with masked characters "*****". A blue "Login" button is at the bottom of the box. The browser window title is "INTERide".

Figure 4.2.1. Login & Registration

In Figure 4.2.1, the login and registration interface of the INTERide Ride Booking System is presented. The design demonstrates the application's emphasis on secure authentication and user-friendly interaction. During testing, the login module correctly validated credentials, prevented duplicate usernames, and applied password hashing to protect sensitive information. These outcomes confirm that the system's account management functions operate reliably and align with best practices in information security. The interface also reflects usability considerations, with clear input fields, responsive error handling, and a straightforward navigation flow. By integrating both functional accuracy and user-centered design, the login and registration component supports the overall objective of creating a stable and trustworthy ride-booking platform.

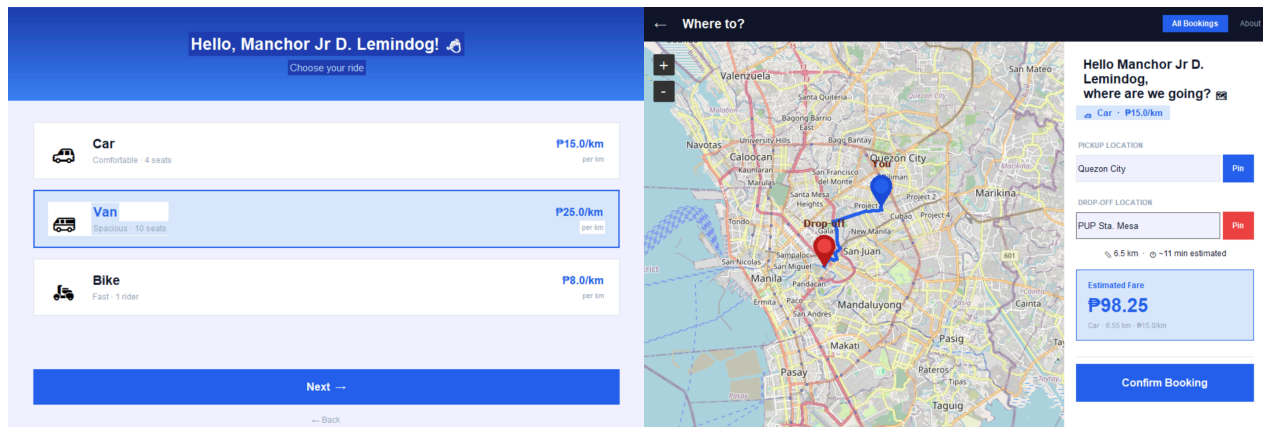


Figure 4.2.2. Fare Calculation

In Figure 4.2.2, the fare calculation process is illustrated. The system dynamically computed travel costs based on the selected vehicle type and the estimated distance between pickup and drop-off points. During testing, the algorithm consistently produced accurate fare breakdowns, including base fare, per-kilometer charges, and additional components such as tolls or surcharges. This confirmed that the BookingModel correctly applied rate-per-kilometer logic for cars, vans, and bikes. The results align with the objectives of ensuring transparency in fare computation, similar to the validation performed in the Gdriver project where different vehicle classes were tested for distinct fare outputs.

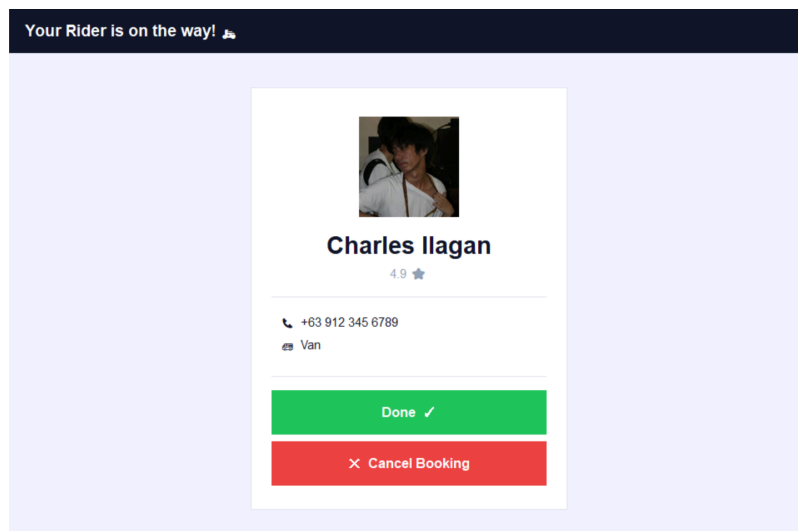


Figure 4.2.3. Booking Confirmation

Figure 4.2.3 demonstrates the booking confirmation stage. Once a ride was successfully created, the system generated a record in the database and displayed a summary of the transaction. Testing verified that booking details such as pickup location, drop-off location, vehicle type, fare, and payment method were accurately stored in the bookings table. The confirmation interface provided immediate feedback to the user, ensuring clarity and reducing the likelihood of duplicate or erroneous bookings. This outcome reflects the system's ability to handle transactional integrity, comparable to the confirmation features documented in the Gdriver RideFun History module.

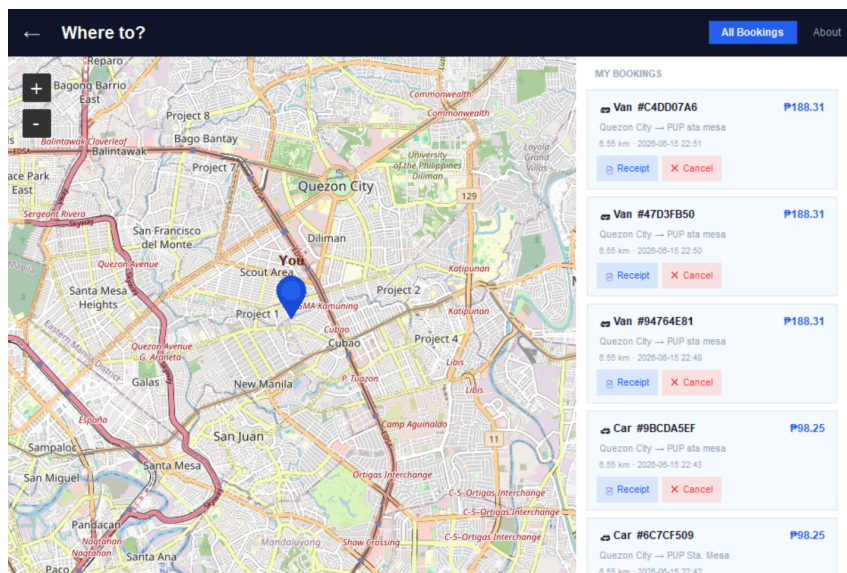


Figure 4.2.4. Ride History

In Figure 4.2.4, the ride history interface demonstrates how the INTERide system maintains a structured record of completed and ongoing bookings, including vehicle type,

route, distance, fare, and transaction time. Testing confirmed that each booking was accurately stored in the database and displayed in the user dashboard, with options to generate receipts or cancel rides. The receipt function produced a clear summary of trip details and payment information, serving as proof of transaction, while the cancellation option updated booking statuses in real time and preserved them in the history log. T

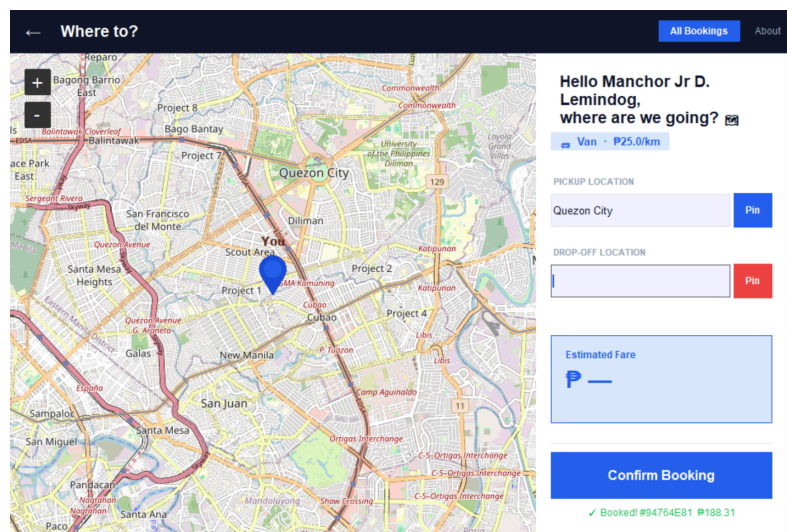
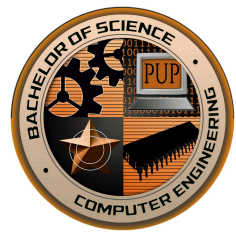


Figure 4.2.5. Map Visualization

Figure 4.2.5 highlights the interactive map widget. During testing, the system accurately displayed pickup and drop-off pins, generated route lines, and updated visual elements dynamically based on user input. The map provided a graphical representation of the journey, improving user experience by making ride details more intuitive. Testing confirmed that the widget correctly responded to location changes and rendered routes without errors. This feature distinguishes INTERide from file-based prototypes like Gdriver by integrating real-time visualization into the booking process, thereby enhancing clarity and user confidence in the system.

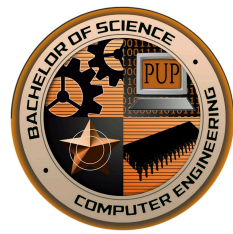


4.3 Analysis of Results

Testing confirmed that INTERide operates as intended, with all major functions performing correctly. The integration of MySQL ensured reliable data persistence, while Tkinter provided a responsive interface. Compared to file-based storage systems like Gdriver (which used JSON), INTERide's database-driven approach offered stronger scalability and reduced risk of data loss.

The fare calculation logic proved adaptable, allowing different vehicle types to be represented realistically. This flexibility mirrors real-world ride-hailing platforms in the Philippines, where commuters select transport modes based on distance, cost, and passenger capacity.

Overall, the results demonstrate that INTERide is a functional prototype capable of simulating real-world ride-booking operations. It validates the application of Object-Oriented Programming principles in building transport management systems and highlights the relevance of academic projects in addressing everyday commuting challenges



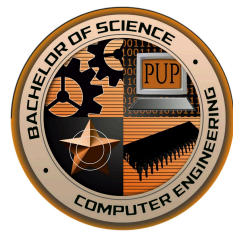
III. CONCLUSION

4.4 Summary of Findings

The testing and evaluation phase validated the overall functionality, usability, and reliability of the INTERide Ride Booking System. Developed as a desktop application using Python and the Tkinter framework, this project demonstrates the practical application of software engineering principles in addressing real-world operational and urban transit problems. By utilizing the foundational pillars of Object-Oriented Programming (OOP), the system achieves a modular and highly reusable system. Specifically, structural testing confirmed a successful OOP implementation where shared attributes cleanly map from a base Vehicle class down to specialized Car, Van, and Bike subclasses. Furthermore, encapsulation effectively secures internal system data by keeping properties private and read-only, while polymorphism enables dynamic cost calculations across different vehicle modes, exemplified by the Car subclass applying a conditional 10% discount profile for journeys exceeding 10 kilometers.

In terms of operational performance, the application ensures high mapping and tracking accuracy by moving away from simple straight-line distance approximations. The integration of the OSRM API alongside the geopy library allows the system to pinpoint locations and calculate authentic road-network distances and journey durations. These routes are then dynamically rendered onto an interactive canvas powered by tkintermapview and OpenStreetMap. For data persistence and reporting, completed bookings are instantly captured as distinct objects and saved sequentially into a local `interide_bookings.json` text file, while the integration of the `python-docx` library automates the export of cleanly formatted, professional Word document receipts to ensure transactional accountability. Finally, user-centric deployment testing verified that the self-installing launcher (`run.py`) successfully automates dependency checks using Python's internal modules, drastically reducing manual configuration and ensuring a seamless system setup.

In conclusion, INTERide establishes the relevance of building practical applications to address contemporary transit needs, providing both its technical validity and social importance for modern commuters.



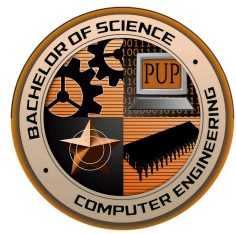
4.5 Recommendations

To advance the INTERIde platform beyond its current desktop-based prototype stage and prepare it for broader utility or production deployment, the following future developments are recommended:

- **Upgrade to a Multi-User Storage System:** Right now, the app saves trip records into a single text file on the computer, which only works for one person at a time and can easily be edited or deleted by mistake. The system should be upgraded to a professional database system so that hundreds of commuters can safely book rides at the exact same second without the app slowing down or mixing up data.
- **Integrate User Authentication and Secure Hashing:** Future updates should incorporate dedicated user profile models featuring secure, password-hashed login credentials. This will ensure that individual trip histories and saved booking records are fully isolated and protected against unauthorized access.
- **Incorporate Live Cashless Payment Gateways:** While the application conceptually supports both cash and GCash selections to match local consumer production-ready financial payment APIs (such as PayMaya, GCash, or PayPal wrappers) would enable real-time electronic transaction processing directly within the app.
- **Deploy a Live Driver-Matching and Status Loop:** Expanding the backend logic to include a dedicated "Driver" module would elevate the realism of the simulation. This layer should calculate real-time driver proximity, simulate ride acceptance workflows, and provide multi-step status updates like "Driver Arriving," "In-Transit," and "Completed."
- **Implement Post-Trip Driver Rating Frameworks:** Introducing an interactive feedback panel directly after the receipt generation step would allow passengers to grade their ride experience. Storing these metrics within the system would provide data accountability and lay the groundwork for driver performance algorithms.
- **Rebuild as a Universal Mobile Application:** Since actual commuters book rides on their phones while on the go, the system should be rebuilt into a mobile app. Using a universal coding setup will allow the team to write the code just once and deploy it



**Republic of the Philippines
Polytechnic University of the Philippines - Sta. Mesa
Department of Computer Engineering
College of Engineering and Architecture**



directly to both Android phones and iPhones, making the system accessible to any commuter in the Philippines.